CodeAlpha Cybersecurity Internship

Task 1 — Basic Network Sniffer

Prathamesh Tikle

CodeAlpha Cybersecurity Internship

March 2026

GitHub: CodeAlpha_NetworkSniffer

1. Objective

Build a Python-based network sniffer capable of capturing live network packets, analysing their structure, identifying protocols, and displaying useful information such as source/destination IPs, ports, flags, and payloads.

---

2. Tool & Libraries Used

Language	Python 3
Library	Scapy — packet crafting and sniffing
Protocols	IP, TCP, UDP, ICMP, DNS
Interface	wlan0 / eth0 (auto-detect supported)
Usage	sudo python3 network_sniffer.py -i wlan0 -c 100 -p tcp

---

3. Features Implemented

Feature	Description
Protocol Detection	Identifies TCP, UDP, ICMP, DNS packets automatically
IP Extraction	Displays source and destination IP addresses
Port Extraction	Shows src/dst ports for TCP and UDP packets
TCP Flag Parsing	Decodes SYN, ACK, FIN, RST, PSH, URG flags
DNS Query Display	Extracts and shows queried domain names
Payload Preview	Shows first 60 chars of raw payload (UTF-8 decoded)
BPF Filter Support	Accepts Wireshark-style filters e.g. 'tcp port 80'
Protocol Shortcut	-p flag for quick tcp/udp/icmp/dns filtering
Colour-coded Output	Green=TCP, Yellow=UDP, Red=ICMP, Cyan=DNS
Capture Summary	Prints packet count per protocol on exit

---

4. How It Works

The sniffer uses Scapy's **sniff()** function to capture packets on the specified interface. Each captured packet is passed to the **process_packet()** callback which:

- Checks if the packet contains an IP layer — skips non-IP frames
- Determines the protocol (DNS checked first as it runs over UDP)
- Extracts source/destination IPs, ports, TTL, and packet length
- Decodes TCP flags using bitwise flag inspection

- Extracts DNS query names from DNSQR layer if present
- Decodes raw payload up to 60 characters using UTF-8
- Prints a colour-coded single-line summary to the terminal
- Updates the packet counter dictionary for the summary report

---

5. Usage Examples

Capture all traffic on wlan0:

```
sudo python3 network_sniffer.py -i wlan0
```

Capture 50 TCP packets:

```
sudo python3 network_sniffer.py -i wlan0 -c 50 -p tcp
```

Capture DNS traffic only:

```
sudo python3 network_sniffer.py -p dns
```

Filter by host using BPF:

```
sudo python3 network_sniffer.py -f 'host 8.8.8.8'
```

Capture HTTP traffic (port 80):

```
sudo python3 network_sniffer.py -f 'tcp port 80'
```

---

6. Sample Output

```
[17:40:12.345] TCP 10.0.0.5:54321 → 142.250.182.46:443 | TTL:64 Len:74B [SYN]
[17:40:12.346] DNS 10.0.0.5:52134 → 10.0.0.1:53 | TTL:64 Len:78B → Query: www.google.com.
[17:40:12.350] TCP 142.250.182.46:443 → 10.0.0.5:54321 | TTL:118 Len:74B [SYN, ACK]
[17:40:12.351] TCP 10.0.0.5:54321 → 142.250.182.46:443 | TTL:64 Len:54B [ACK]
[17:40:12.400] UDP 10.0.0.5:68 → 255.255.255.255:67 | TTL:64 Len:342B
[17:40:12.500] ICMP 10.0.0.5 → 8.8.8.8 | TTL:64 Len:84B
```

---

7. Protocol Analysis

Protocol	Layer	Key Fields Extracted	Security Relevance
TCP	Transport	Src/Dst ports, flags (SYN/ACK/FIN/RST), seq/ack	Session tracking, port scan detection
UDP	Transport	Src/Dst ports, length	DNS exfiltration, UDP flood detection
ICMP	Network	Type, code, payload	Ping sweep, ICMP tunnel detection
DNS	Application	Query name, record type (A/AAAA/HTTPS)	DNS poisoning, C2 beacon detection

---

8. Ethical & Legal Notice

Network sniffing must only be performed on networks you own or have explicit written permission to monitor. Capturing traffic on public or corporate networks without authorisation is illegal under the IT Act 2000 (India) and equivalent laws globally. This tool was developed and tested in an isolated local lab environment.